RED-BLACK TREE

Interactive insertion, rotations and two-child deletion

Design specification | Revision 2 | 3 November 2026
Target: **ChrisWhealy/svg-dom-graph**, default branch **main**, reviewed at commit d988a658eaa8. The design is a proposed implementation, not an assertion that the new demo exists.

Purpose. Provide a repeatable, inspectable SVG graph that constructs a real red-black tree, performs both rotation directions, then deletes a node with two children. The implementation must demonstrate that parent-child connectors refer to the correct logical nodes after topology changes and that selection and keyboard focus survive scene replacement.

Design choice	Baseline decision
Fixture	Insert **10, 5, 1, 15, 20, 12**; delete **15**.
Observable operations	Right-rotate at 10 (insert 1); left-rotate at 10 (insert 20); on deletion, left-rotate at 10 and right-rotate at 20.
Rendering model	Pure-Rust event trace; rebuild a complete Scene for each structural snapshot. Preserve identity by key, not by `NodeId`.
No new public mutation API	Current public Scene supports adding graph elements and moving nodes, but not arbitrary edge removal or retargeting. Do not fake a rotation by relabelling nodes.
Selection and focus	Store `chosen_key` (user intent); derive selection and roving focus from each snapshot. No navigation history or scene-local IDs are persisted.

1 Objectives and scope

- **Construct:** start from an empty tree and expose the six ordered insertions, their recolourings and rotations; no hard-coded picture may stand in for the tree algorithm.
- **Rotate:** display a genuine change to parent/left/right links. Every node retains its key and immutable algorithmic identity; the rotated parent, promoted child and transferred subtree are all visible.
- **Delete:** delete key 15 only when it has both children 10 and 20; use its in-order successor 20 and run the black-successor deletion fix-up.
- **Verify:** test tree invariants at settled commits, actual parent-child connector geometry at every structural frame, selected-key resolution across reverse/seek, and browser keyboard focus.
- **Accessible:** provide explicit event explanations, an announcement for changes, non-colour indicators of node colour, visible selected/focused state, and usable keys/controls.

Out of scope: free-form editing, duplicate keys, arbitrary deletion, randomised layout, force-directed physics, user dragging, live node movement, and changes to the public graph model. A separate later study could cover in-place edge retargeting; this demo tests topology by rebuilding real Scene edges.

Revision 2 decisions

This revision closes seven pre-build gaps: history-free chosen/effective selection, explicitly disabled drag, Phase 0 straight-circle endpoint validation, three named jump targets, fixed playback timing, a tree/treeitem keyboard contract, and a parent-aware Option/NIL deletion cursor. These are binding contracts for Phases 0–4.

2 Existing API and integration constraints

Repository inspection shows `Scene::add_circle_node_with`, `Scene::add_edge_with`, `Scene::node_rect`, `Scene::move_node`, `Scene::set_focus` and `Scene::set_edge_focus`. `Scene::set_selection` is for data-node cell selection, not for selecting a tree vertex. Connector geometry is automatically recomputed when a node moves within one scene. The internal graph rollback-only removals do **not** provide general public topology deletion.

Follow the gallery integration pattern: add `demo-app/src/red_black_tree/`, register `panel-red-black-tree` in `init_panel`, and add `demo/panels/11-graph-algorithms/04-red-black-tree.html`. Keep a live strong Scene handle for the displayed graph and release the old handle only after a successful replacement. Controls reside outside the replaceable SVG subtree.

Phase 0 gates (resolve before implementing the renderer)

- Confirm a staged SVG can remain DOM-connected and measurable without being visible or tabbable (`visibility:hidden`, not `display:none`), then atomically replace the displayed root, following established scene-rebuild practice.
- Confirm a reliable way to expose per-key accessible/focusable hit targets in the new SVG without depending on private node-handle or connector-handle ordering. A semantic overlay keyed by `data-rbt-key` is acceptable; relying on `nth-child` is not.
- Confirm the browser can enumerate exactly the library-generated connector paths (for example, the marked path elements in the graph content layer), excluding marker definitions and the semantic overlay; no DOM-child-order or private EdgeId lookup is allowed.
- Confirm the **actual browser geometry** of the default Straight connector between two circular nodes, including path direction, measured fitted radii, CTM conversion and endpoint precision. The reviewed Rust route uses `ellipse_straight_anchor → boundary_towards` with no forced side or clock, but a live two-circle CDP probe must validate the oracle before choosing its tolerance.
- Confirm SVG `role=tree` and `role=treeitem` elements are correctly keyboard-focusable and announced in supported browsers, with one roving tab stop. Fall back to a synchronised HTML tree if this cannot be established.

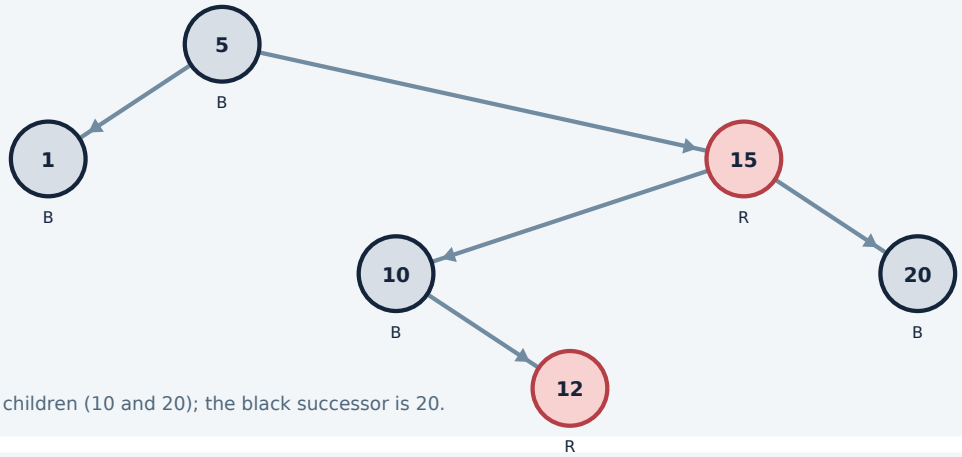
3 Canonical fixture and expected results

Use a classic bottom-up red-black tree with `Option<NodeKey>` children. Absent children represent conceptual black NIL leaves; there are **no allocated NIL nodes and no NIL parent pointers**. Keys are unique u16 values. Each node retains its identity across rotations and transplants. These are golden settled states. B = black, R = red.

Checkpoint	Root / structure	Mandatory observation
Insert 10	10 B; singleton	Empty-to-root creation.
Insert 5	10 B; left 5 R	A real first parent-child connector.
Insert 1	5 B; left 1 R; right 10 R	**Right rotation at 10.** Edges become 5→1 and 5→10.
Insert 15	5 B; 1 B; 10 B with right 15 R	Uncle-red recolouring; no rotation.
Insert 20	5 B; 1 B; 15 B with 10 R and 20 R	**Left rotation at 10.** 5→10 and 10→15 disappear; 5→15 and 15→10 appear.
Insert 12	5 B; 1 B; 15 R with 10 B (right 12 R) and 20 B	Deletion fixture reached. Node 15 has two children; successor 20 is **black**.

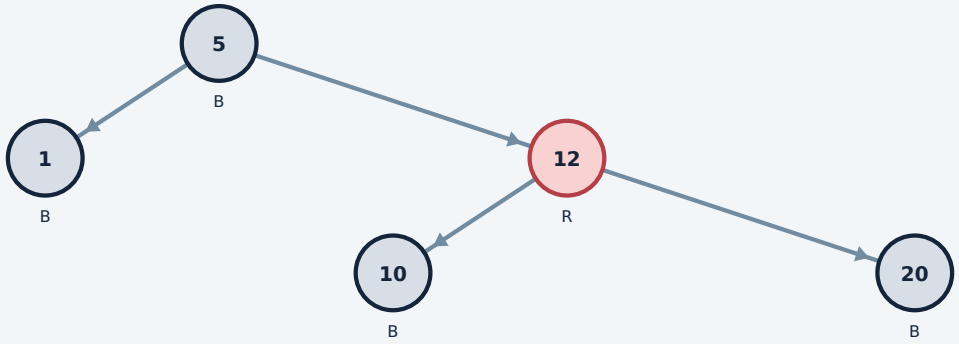
Checkpoint	Root / structure	Mandatory observation
Delete 15	5 B; 1 B; 12 R with 10 B and 20 B	Successor 20 replaces 15, followed by **left rotation at 10** and **right rotation at 20** during delete fix-up.

Before deletion: after all six insertions



Node 15: two children (10 and 20); the black successor is 20.

After deletion and both fix-up rotations



Node 15 is gone; 20 is the original 20, not a relabelled 15.

Precise delete milestones

D0: 15 has children 10 and 20. **D1:** choose successor 20 (black, with no left child). **D2:** move the original node 20 to 15's position, copy 15's red colour and remove node 15. The missing right child of 20 carries black-deletion debt: `x=None, x_parent=Some(20), x_side=Right`. **D3:** mirrored fix-up case 3 left-rotates sibling 10, bringing 12 above it. **D4:** mirrored case 4 right-rotates 20. The final subtree is rooted at red 12, with black 10 and 20; the missing NIL is described in the event text, never drawn.

4 Algorithm contract

Implement the insertion and deletion transformations in plain Rust, independent of DOM, Scene, timers or rendering. Use conventional CLRS-style bottom-up cases and mirrored cases; compute the successor by minimum of the right subtree. Deletion must **transplant the original successor node** rather than copy its key into the deleted node. This preserves the identity that connector and focus checks are designed to track.

```
enum Colour { Red, Black }
struct Node { id: NodeKey, key: u16, colour: Colour,
              parent: Option<NodeKey>, left: Option<NodeKey>,
              right: Option<NodeKey> }
struct Tree { root: Option<NodeKey>, nodes: BTreeMap<NodeKey, Node> }
  // Absent children are black NIL; no NIL node is allocated.
  enum ChildSide { Left, Right }
  struct FixupCursor { x: Option<NodeKey>,
                       x_parent: Option<NodeKey>,
                       x_side: ChildSide }
```

- NodeKey is stable for every surviving node. A key identifies the same object across all rotations, recolourings, predecessor searches and transplants. Never use the scene-local `NodeId` as a permanent identity.
- All parent/child changes in one rotation or one complete transplant are an **atomic structural event** in the trace. Emit a snapshot only after reciprocal parent links, root replacement and transferred subtree links are correct.
- Perform ordinary insert-fix-up and delete-fix-up recolouring as explicit events. Each settled insertion and deletion must satisfy: black root; black NIL leaves; no red parent with red child; identical black-height on all root-to-NIL paths; strict BST order; no cycles or unreachable nodes.
- Transitional frames may have a red-red conflict or black-height imbalance while the textbook fix-up is incomplete. Label these **transition**, not a valid committed red-black tree. At every transition still require coherent parent/child references, acyclicity and sorted in-order keys.
- Deletion fix-up must take `x`, `x_parent` and the missing child side explicitly. An absent `x` has no parent pointer, and `None == parent.left/right` is not enough to infer its side when both links are absent. For this fixture the cursor starts at `(None, 20, Right)`. Keep the cursor current through rotations; never dereference a NIL parent.
- Reject duplicate keys at the solver boundary. The fixture has no invalid inputs. Independently test empty/singleton, missing-key, root deletion, two-child deletion with direct and deeper successors, and both left/right missing-child cursors to cover `x=None` fix-up.

5 Event trace and snapshot specification

The solver creates one deterministic `Trace`. No DOM action re-runs the algorithm. The projection for a trace index is a pure function of that snapshot and UI preferences. Steps must support forward, reverse, arbitrary seek and Restart, with no cumulative mutation in rendering code.

```
struct Snapshot {
  ordinal: usize, phase: Phase, event: Event,
  root: Option<NodeKey>, nodes: Vec<NodeSnapshot>,
  edges: Vec<(NodeKey, NodeKey)>,
  active: Vec<NodeKey>, description: String,
  settled: bool,
}
enum Event {
  Insert(NodeKey), Recolour(Vec<(NodeKey, Colour)>),
  RotateLeft(NodeKey), RotateRight(NodeKey),
  ChooseSuccessor { deleted: NodeKey, successor: NodeKey },
  Transplant { deleted: NodeKey, successor: NodeKey },
  DeleteFixupCase { case: u8, mirrored: bool },
  CompleteInsert(NodeKey), CompleteDelete(NodeKey),
}
```

At minimum record: empty, every linked insertion, every recolour, all four fixture rotations, successor selection, completed transplant, fix-up milestones and the final settled state. A deletion-fix-up event includes the cursor's x, x_parent and side and its narrated missing-NIL position. No cosmetic frame may claim a topology change while reusing obsolete edges.

Golden trace checks locate events by phase, semantic kind and pivot, never by fragile indices. Construction: `Right(10)` before `Left(10)`. Deletion: `ChooseSuccessor(15,20)` → `Transplant(15,20)` → `Left(10)` → `Right(20)`. Every rotation must actually change topology. Jump labels and destinations are defined in Section 8.

6 Rendering, layout and connector contracts

6.1 Nodes, positions and colours

Use `Scene::add_circle_node_with` with a readable fill for black and red nodes (slate and pale red respectively), and append "B" or "R" to the visible key label so information is not encoded by colour alone. Node radius is at least 28 SVG units; labels may enlarge the radius via the existing ellipse fit logic, so read the actual `node_rect` afterward. If a dark fill is desired instead, Phase 0 must establish a reliable light-text rendering method.

Use a fixed key universe `[1,5,10,12,15,20]`; centres are `x = 100 + rank*106` and `y = 96 + depth*105` in an approximately 760 × 540 viewBox. Recalculate depth per snapshot and measure any radius increase through `node_rect`. No drag or user-position offsets are stored. **Every Previous, Next, Restart, jump, seek or playback step resets all positions to this fixed layout**; deletion leaves key 15's horizontal slot vacant.

6.2 Authoritative connections

For every snapshot, derive edges exclusively from the algorithm's reciprocal parent/child links, one downward directed edge per non-root node. Create all circle nodes first, map `NodeKey → NodeId` for that scene only, then use `Scene::add_edge_with(parent_id, child_id, Straight)`. Use the library's own paths; do not draw a second set of visual imitation links.

Acceptance: with N displayed nodes there are exactly N−1 paths (or zero when N=0); each non-root key has exactly one incoming edge; the root has none; each edge's logical endpoints match the snapshot; no path terminates at the removed key 15. A snapshot rebuild will naturally create fresh `NodeIds` and `EdgeIds`; never compare these across scenes.

6.3 Test actual path geometry

Browser tests enumerate the actual library `path[marker-end]` connectors, not overlay strokes. For each path use `getPointAtLength(0)` and `getPointAtLength(getTotalLength())`, convert by its CTM into the root SVG user coordinates, then compare oriented start/end points against the expected parent/child circles. Match the paths to the expected pairs as a one-to-one unordered assignment; count errors, reversed pairs, duplicates and obsolete links as failures.

Geometry oracle: source review confirms that an unpinned Straight edge between Scene circles uses `boundary_towards` from each centre towards the opposite centre. With measured centre C, fitted radius r and opposite centre Q, the expected endpoint is `C + r*(Q-C)/|Q-C|`. Phase 0 must prove this in a browser on horizontal and diagonal pairs, checking start/end direction, radius fitting and CTM conversion. Only after the probe passes should tests adopt the proposed maximum 1.5 SVG-unit error; an oracle discrepancy is a blocker, not permission to loosen the tolerance.

Drag decision: `make_draggable` and all live `move_node` probes are deliberately excluded from this interactive demo. The separately positioned `data-rbt-key` hit targets therefore cannot drift away from the circles. The baseline verifies

real connectors across topology rebuilds, not live edge retargeting or drag; those require a separate demo with overlay realignment.

7 Selection, keyboard focus and accessibility

Selection is **user intent**, not the visible replacement node. Persist `chosen_key: Option<NodeKey>` and derive `effective_key = resolve(chosen_key, snapshot)` afresh on every step. Never overwrite `chosen_key=Some(15)` merely because 15 is absent after deletion. Reverse/seek before deletion then automatically selects 15 again; **no selection-history stack** exists. `Scene::set_focus` draws only a visual ring, not actual keyboard focus; `set_selection` is for data-cell selection, not this tree.

7.1 Pure resolution rules and navigation intent

```
fn resolve(chosen: Option<NodeKey>, s: &Snapshot) -> Option<NodeKey> {
  match chosen {
    None => None,
    Some(k) if s.has_key(k) => Some(k),
    Some(15) if s.delete_started() && s.has_key(20) => Some(20),
    Some(_) => s.root.or_else(|| s.minimum_key()),
  }
}
// Keep chosen_key unchanged until an explicit user selection.
// Roving-key intent has its own stable key and uses a
// corresponding resolve (with initial root fallback).
```

Selection and keyboard focus are distinct. Persist `roving_key_intent: Option<NodeKey>` independently of `chosen_key`; compute the current roving key from the snapshot using the same absent-key replacement rule, except an unset roving intent starts at the current root. Keep the active focus location separately as graph node, stable control action or outside page. A programmatic focus restore must **not** overwrite these intents via its focus event. Actual user arrow/pointer focus changes update roving intent; Enter/Space/click selection changes chosen_key.

- If chosen_key is None, effective selection is None. If the requested key exists in the current snapshot, it wins; otherwise key 15 after its deletion resolves to original successor 20. For another not-yet-created or absent key, fall back to the current root, then minimum key, else None on the empty tree. A direct seek and repeated reverse must give exactly the same result.
- When 15 was chosen, stepping past deletion highlights 20 while retaining chosen_key=15. Going backward to any snapshot containing 15 highlights 15 again. If the user explicitly selects 20 after deletion, chosen_key becomes 20, so reversing does not select 15.

7.2 Accessible tree structure and keyboard scheme

Create an SVG semantic hit layer above the Scene circles with `role=tree` and one `role=treeitem`/`data-rbt-key` target per visible key. Use nested `role=group` descendants matching the algorithmic parent/child hierarchy, with independently positioned transparent hit circles so nested DOM does not shift geometry. Each treeitem has `aria-level=depth+1`, truthful `aria-posinset`/`aria-setsize`, `aria-selected` and a name such as “12, red, right child of 10”. Use `aria-expanded=true` only on nodes with shown children; there is no collapse feature. Test actual screen-reader tree exposure in Phase 0.

Exactly one treeitem has `tabindex=0`; all others have `-1`. Tab enters or leaves the tree once. Up/Down step through in-order keys; Home/End go to minimum/maximum; Left goes to parent; Right goes first to left child, otherwise right child, otherwise does nothing; Enter/Space chooses the focused key. **The Right key is not a “go to right child” command:** where a node has two children, reach its right child via Up/Down. State that limitation in a visible keyboard-help sentence and test it on a node with two children. Handled navigation keys prevent default scrolling; unrelated keys do not.

Visible help copy: “Tab to enter the tree. Up/Down move in sorted key order; Home/End go to the first/last key; Left goes to the parent; Right goes to the first available child (left before right); use Up/Down to reach the other child; Enter/Space selects.”

Attach hit targets from freshly measured `Scene::node_rect` positions after each rebuild. Since user drag is disabled, every target remains aligned to its circle. Ensure hit targets receive clicks without blocking controls, and verify their centres agree with the rendered node centres after each of the four rotations and the transplant. Style effective selection separately from browser focus, and expose both without relying on node colour alone.

7.3 Focus restoration contract

Previous active focus	Post-commit behaviour
Graph key surviving	Restore browser `activeElement` to that exact key’s newly built treeitem, even when its parent and coordinates change.
Graph key 15 deleted	Restore actual focus to treeitem 20, but retain roving_key_intent=15. Reverse across deletion returns focus to 15 unless the user navigated elsewhere.
Graph node not yet inserted	Use the deterministic root/minimum fallback, or the tree container when no key exists; announce fallback.
Next/Previous/Restart, jump or seek control	Keep focus on the same stable control outside the replaced graph. Repeated keyboard activation must remain possible.
Outside the demo	Do not steal browser focus; only re-project the effective selection and roving tab stop.

Capture focus by stable key or action before replacing the graph, restore it only after new nodes are attached and accessibility attributes set, and never save an old `NodeId` or stale DOM element. The chosen selection intent must not change during automatic restore, play, seek, or step. On intentional keyboard navigation, update the roving key to the newly reached node.

Expose the root, size, chosen/effective selection and parent/children in readable text. Event announcements use `aria-live=polite`, and transient black-height/red-red repair frames must not be announced as settled valid trees. The selected ring and visible keyboard focus indicator are distinct and readable on both fill colours.

8 Interaction and transactional redraw

Controls (all stable DOM siblings of the replaced graph): **Restart, Previous, Next, Play/Pause**, an accessible seek range from `0..=trace.last()` (0 = empty tree), and three named jumps: **Right(10)** seeks the construction `RotateRight(10)`; **Left(10)** seeks the construction `RotateLeft(10)`; **Delete 15** seeks the first `ChooseSuccessor(15,20)` event, not the completed deletion. Differentiate the construction Left(10) from deletion Left(10) by phase. Give each control a stable `data-rbt-action`.

Playback is fixed at **2 steps per second (500 ms per successfully committed step)**; there is **no speed control** in this revision. The first Next advances from empty to the first insertion event. Boundary presses are no-ops but announce “Already at the first/last step”. Playback stops at the last settled frame and schedules a new tick only after a successful commit.

A step builds a candidate SVG and Scene at the **fixed per-key/depth positions** in a connected, non-displayed staging host; it never retains previous coordinates. Keep the staging host inert/untabbable, verify model/edge maps, save chosen_key, roving intent, active focus location and viewport preferences, then replace the graph subtree. Rebind new scene IDs and overlay treeitems, re-project effective selection, and restore actual focus after attachment. Stable controls remain outside the replaced subtree.

If candidate construction fails, keep the old scene and committed index. On DOM swap failure restore the old root as far as possible and report the error. If post-swap styling or focus fails, report the fault and mark the projection **dirty**: repair must run on the next interaction even if that interaction is a boundary no-op, before processing its requested action. Advance the playback epoch on **every** stop (Pause, manual navigation, seek/jump, error, teardown and final completion); never leave an outstanding old tick able to commit a later snapshot.

Every user action commits at most one logical snapshot; callbacks schedule only after successful commits. Seek/reverse read immutable snapshots, not inverse mutations. No drag listeners or debug movement switches are shipped, because a moved Scene circle would otherwise separate from its semantic hit target. Scene rebuild restores the specified fixed layout on every step.

9 Architecture and ownership

File / layer	Responsibility
`algorithm.rs`	Tree and bottom-up insert/delete logic; no browser dependencies; stable node identity; invariant checker.
`trace.rs`	Deterministic snapshots/event descriptions; golden fixture assertions; semantic milestone lookup.
`layout.rs`	Fixed-key x slots, depth-dependent y positions, width/radius constraints and status/control placement.
`render.rs`	Build Scene circles/edges, keep per-frame key→NodeId and pair→EdgeId maps, and project focus rings.
`accessibility.rs`	Semantic key hit targets, roving tabindex, keyboard rules, announcements and focus capture/restore.
`controller.rs`	Index/playing/epoch/chosen_key/roving intent/active focus location; pure resolve, fixed-speed stepping, seek, bounds and repair.
`mod.rs`	Gallery entry point, DOM discovery, event listener lifetimes, teardown and source listing.
`cdp/red_black_tree.rs`	End-to-end DOM geometry, keyboard continuity, scene swap, buttons and controlled failure tests.

Model objects and snapshots are pure data; the projection alone owns per-scene `NodeIds` and `EdgeIds`. Strongly own the live Scene without listener cycles. Reuse `demo_common::focus` for controls, but derive key targets from the current snapshot. No history stack, drag state or user-position cache is required.

10 Test and acceptance matrix

Layer / priority	Required automated assertion
Solver / P0	All settled states have exact keys, reciprocal links, sorted traversal, black root, no red-red, uniform conceptual NIL black-height and reachability.
Fix-up cursor / P0	Option-child deletion uses explicit x, x_parent and side; fixture starts (None,20,Right); test absent left/right children and deeper successor cases without NIL dereference.
Trace / P0	Four rotations in order: insertion Right(10), insertion Left(10), deletion Left(10), deletion Right(20); successor 20 is black; all topology events change correct links.
Transplant / P0	Original 20 survives, 15 is removed; final pairs exactly 5→1, 5→12, 12→10, 12→20. No copied 15 label/identity.
Snapshot / P0	Every structural frame reciprocal and acyclic; transient colour repairs labelled; reverse and direct seek match pure projections.
Phase 0 geometry / P0	Horizontal and diagonal two-circle probe matches actual Straight boundary rule and path orientation after CTM conversion; justify tolerance before shipping endpoint tests.
CDP connectors / P0	After every structural event: N−1 genuine marked paths, one-to-one directed assignment to expected pairs, endpoint error no more than Phase 0 tolerance; obsolete paths absent.
Overlay / P0	No drag/move handlers. Every semantic treeitem target is centred on its Scene circle; each step resets fixed x/depth coordinates.
Selection / P0	chosen=15 resolves to 20 after delete but stays 15 as intent; reverse/seek restores 15; explicitly choosing 20 suppresses that restoration; no history.
Focus / P0	Focus on surviving 10 or 20 follows identity; focus on 15 transfers to 20 and back on reverse without user navigation; focused Next stays Next.
Keyboard / P0	One roving treeitem; aria-level/posinset/setsize/selected accurate. Right picks left child first; Up/Down can reach right child; no focus theft.
Controls / P0	Three jumps hit Right(10), Left(10) in construction and ChooseSuccessor(15,20); seek range 0..last; fixed 500 ms Play ticks and bounded status.
Fault repair / P1	Candidate failure leaves prior state; swap error rolls back; dirty repairs on next action including boundary no-op; stale epoch tick is ignored.
A11y / P0	Accessible tree/treeitem/group roles, explicit red/black labels, keyboard help note, distinct focus vs selection and live event explanation.

Golden geometry assertion: inspect the real rendered paths after the final delete and compare them against `[(5,1),(5,12),(12,10),(12,20)]`. The same tree shape drawn with key labels swapped is **not** accepted: the test reads the semantic key-to-centre mapping and checks each path's actual endpoints. Similarly, after inserting 20 the required pairs are `[(5,1),(5,15),(15,10),(15,20)]`.

11 Delivery phases and release gate

Phase	Completion gate
0 — API and DOM spike	Prove staging/swap, SVG tree/treeitem focus, overlay alignment, real connector discovery and two-circle Straight endpoint oracle. Fix tolerance only after the browser probe.
1 — Pure solver	Six insertions, explicit NIL fix-up cursor and two-child deletion produce golden states; invariant tests include both successor arrangements.
2 — SVG scene	Rebuild genuine circle/edge topology, no dragging, fixed positions; add aligned semantic overlay and pass connector endpoint oracle at each structural event.
3 — Controls / focus	chosen/effective selection, roving intent, stable keyboard focus, three named jumps, seek and fixed 500 ms playback, rollback and teardown.
4 — CDP / hardening	All P0 tests pass; P1 transactional fault injection passes or is explicitly deferred; visual snapshots and gallery/source integrations complete.

Release criteria: (1) deterministic six-insert/delete trace and explicit NIL fix-up cursor; (2) confirmed browser connector geometry oracle and no stale/wrong path after topology changes; (3) reversible chosen_key=15 selection with effective 20 after deletion; (4) tree/treeitem keyboard access and identity-preserving focus; (5) clearly named jumps, fixed-speed playback and reliable transactional repair; and (6) tests pass. Public edge mutation APIs and dragging are not part of this design.

12 Checked repository references

Primary source links, all on the reviewed default-branch revision. These are implementation references, not external algorithmic dependencies.

- Basic static tree example
- Graph append-only / rollback removal contract
- Public circle node creation
- Public move_node / set_focus
- Straight ellipse connector anchor rule
- Focus restoration helper
- Full scene rebuild precedent
- Structured keyboard/focus precedent